

Python syntax guide

Every command this course teaches, with what it does and an example you can try. Nothing here is the answer to an activity; it is what to look up while you work one out.

The groups are in the order the course teaches them, and each one says which lesson it belongs to. The same list is on screen behind the **Syntax reminder** button, which only ever shows you as far as the lesson you are on. 99 entries in all.

Output and variables — lesson 1

`print(something)`

Shows one line on the screen; text to show goes in quotation marks.

```
print("Good morning")
```

shows **Good morning**

`print(one, two)`

Shows two values on the same line, with one space put between them. The other way is to join them yourself with + and str(), which is what the lessons do, because then you decide where every space goes.

```
print("socks", 3)
```

shows **socks 3**

`variable = value`

Stores a value under a name, throwing away whatever that name held before.

```
colour = "amber"
print(colour)
```

shows **amber**

`NAME = value`

A constant: a variable written in capitals to show that it must never change.

```
SPEED_LIMIT = 30
print(SPEED_LIMIT)
```

shows **30**

`"text" + "text"`

Joins two pieces of text into one, adding no space of its own.

```
print("sun" + "flower")
```

shows **sunflower**

`# a note`

A comment: everything after the # is for people to read and Python ignores it.

```
print("ready") # this part is ignored
```

shows **ready**

Reading input, and types — lesson 1

`input(prompt)`

Waits for the user to type a line and hands back what they typed, always as text.

```
animal = input("Animal? ") # the user
types otter
print("You chose", animal)
```

shows **You chose otter**

`int(text)`

Turns text into a whole number, so that sums can be done with what was typed.

```
age = int(input("Age? ")) # the user
types 14
print("Next year", age + 1)
```

shows **Next year 15**

`float(text)`

Turns text into a number that is allowed a decimal part.

```
mass = float(input("Mass? ")) # the user
types 2
print("Mass is", mass)
```

shows **Mass is 2.0**

`str(number)`

Turns a number into text, so that + can join it onto other text.

```
legs = 8
print("Legs: " + str(legs))
```

shows **Legs: 8**

Arithmetic — lesson 2

`a + b a - b a * b`

Adds, takes away and multiplies two numbers.

```
print(7 + 2, 7 - 2, 7 * 2)
```

shows **9 5 14**

`a / b`

Divides, and the answer always keeps a decimal part, even when it goes exactly.

```
print(9 / 2, 8 / 2)
```

shows **4.5 4.0**

a // b

Divides and throws the fraction away, so it says how many whole lots fit (OCR calls it DIV).

```
print(17 // 5)
```

shows **3**

a % b

Gives what is left over after dividing (OCR calls it MOD).

```
print(17 % 5)
```

shows **2**

a ** b

Raises a number to a power, and a power of 0.5 is the square root.

```
print(2 ** 5, 49 ** 0.5)
```

shows **32 7.0**

round(number, places)

Rounds a number, keeping as many decimal places as you ask for, or to a whole number if you ask for none.

```
print(round(8.476, 2))
print(round(8.476))
```

shows **8.48 / 8**

random.randint(low, high)

Picks a whole number from low to high, and both of those ends can come up.

```
import random
roll = random.randint(1, 6)    # 1, 2, 3, 4,
                               # 5 or 6
```

Deciding with if — lesson 3

if condition:

Runs the indented lines underneath only when the condition is true.

```
if 8 > 3:
    print("bigger")
```

shows **bigger**

else:

Runs its own indented lines when the if above it was false.

```
if 4 > 9:
    print("bigger")
else:
    print("smaller")
```

shows **smaller**

elif condition:

Asks a further question, but only when every if and elif above it was false.

```
mark = 55
if mark >= 70:
    print("high")
elif mark >= 40:
    print("middle")
```

shows **middle**

Comparing and combining — lesson 3

a == b a != b

Asks whether two values are the same, or are not the same; one = would store a value instead of asking.

```
print(5 == 5, 5 != 5)
```

shows **True False**

a < b a > b a <= b a >= b

Asks which of two values is the smaller or the bigger, and <= and >= allow them to be equal.

```
print(4 < 4, 4 <= 4)
```

shows **False True**

condition and condition

True only when both sides are true, and each side needs a full comparison of its own.

```
print(5 < 10 and 3 > 1, 5 < 10 and 3 > 9)
```

shows **True False**

condition or condition

True when at least one of the two sides is true.

```
print(2 > 7 or 3 == 3)
```

shows **True**

not condition

Turns true into false, and false into true.

```
print(not 3 == 3)
```

shows **False**

Counting loops — lesson 4

for i in range(stop):

Repeats the indented lines, with i counting 0, 1, 2 up to one less than stop.

```
for i in range(3):
    print(i)
```

shows **0 / 1 / 2**

for i in range(start, stop):

Counts from start and stops before stop, so to finish on 10 you write range(1, 11).

```
for i in range(1, 4):
    print(i)
```

shows **1 / 2 / 3**

for i in range(start, stop, step):

Counts in steps of your own size, and a step of -1 counts downwards.

```
for n in range(5, 2, -1):
    print(n)
```

shows **5 / 4 / 3**

total = total + value

A running total: set it to 0 before the loop, add to it inside the loop, print it after.

```
total = 0
for n in range(1, 4):
    total = total + n
print(total)
```

shows **6**

Loops that wait for a condition — lesson 5

while condition:

Repeats the indented lines for as long as the condition stays true, so something inside has to change it.

```
n = 3
while n > 0:
    print(n)
    n = n - 1
```

shows **3 / 2 / 1**

while value != sentinel:

Reads one value before the loop and the next at the bottom of it, stopping on the value that means finish.

```
line = input() # the user types 4
while line != "quit":
    print(line)
    line = input() # then quit
```

shows **4**

if value > best:

Keeps the biggest so far: start it low, and replace it whenever something bigger turns up.

```
best = 0
if 6 > best:
    best = 6
print(best)
```

shows **6**

Working with text — lesson 6

len(thing)

How many characters a piece of text holds.

```
print(len("otter"))
```

shows **5**

text[position]

The one character at that position, counting from 0, so the last one is at len() - 1.

```
word = "otter"
print(word[0], word[4])
```

shows **o r**

text[-1]

The last character, without having to work out how long the text is.

```
print("otter"[-1])
```

shows **r**

text[start:end]

A slice: the characters from start up to but not including end.

```
print("anchor"[0:3])
```

shows **anc**

text.upper() text.lower()

Hands back a copy with every letter made capital, or made small, leaving the original as it was.

```
print("Ferry".upper(), "Ferry".lower())
```

shows **FERRY ferry**

word in text

True when the word appears somewhere inside the text, although it does not say where.

```
print("paint" in "wet paint")
```

shows **True**

text.find(word)

The position where the word first appears, or -1 when it is not in there at all.

```
print("wet paint".find("paint"), "wet paint".find("dry"))
```

shows **4 -1**

for ch in text:

Hands you the characters one at a time, starting at the front.

```
for ch in "hi":
    print(ch)
```

shows **h / i**

text = text + more

Builds longer text by joining more onto the end of what is there already.

```
keep = ""
for ch in "a1b":
    keep = keep + ch + "-"
print(keep)
```

shows **a-1-b-**

ord(character) chr(code)

ord() gives a character's number code and chr() turns a code back into a character.

```
print(ord("a"), chr(98))
```

shows **97 b**

Lists — lesson 7

[value, value, value]

Makes a list, which holds many values in one variable.

```
pets = ["cat", "dog"]
print(pets)
```

shows **['cat', 'dog']**

list[position]

The item at that position, counting from 0, exactly as with text.

```
pets = ["cat", "dog", "ant"]
print(pets[1])
```

shows **dog**

list[position] = value

Replaces the item stored at that position and leaves the rest of the list alone.

```
nums = [4, 9]
nums[0] = 7
print(nums)
```

shows **[7, 9]**

list.append(value)

Adds a value onto the end of the list, making it one item longer.

```
pets = ["cat"]
pets.append("dog")
print(pets)
```

shows **['cat', 'dog']**

len(list)

How many items the list holds, so the last position is len(list) - 1.

```
print(len([4, 9, 2]))
```

shows **3**

for item in list:

Hands you each item of the list in turn, with no position numbers needed.

```
for p in ["cat", "dog"]:
    print(p)
```

shows **cat / dog**

for i in range(len(list)):

Counts the positions of the list, so list[i] is the item the loop has reached.

```
pets = ["cat", "dog"]
for i in range(len(pets)):
    print(i, pets[i])
```

shows **0 cat / 1 dog**

total / len(list)

The average: a total shared out between however many items there are.

```
nums = [5, 7, 9]
total = 5 + 7 + 9
print(total / len(nums))
```

shows **7.0**

list.remove(value)

Takes the first item matching that value out of the list.

```
pets = ["cat", "dog", "cat"]
pets.remove("cat")
print(pets)
```

shows **['dog', 'cat']**

del list[position]

Takes out the item at that position, and everything after it shuffles up one place.

```
pets = ["cat", "dog", "ant"]
del pets[0]
print(pets)
```

shows **['dog', 'ant']**

Grids: lists inside lists — lesson 8

[[a, b], [c, d]]

A two-dimensional list: a list whose items are themselves lists, one for each row.

```
grid = [[4, 9], [8, 1]]
print(grid)
```

shows **[[4, 9], [8, 1]]**

grid[row][column]

The value in that row and that column, both counting from 0.

```
grid = [[4, 9], [8, 1]]
print(grid[1][0])
```

shows **8**

grid[row]

One whole row, which is an ordinary list you can print, measure or total.

```
grid = [[4, 9], [8, 1]]
print(grid[0])
```

shows **[4, 9]**

len(grid) len(grid[0])

How many rows the grid has, and how many values are in one row.

```
grid = [[4, 9, 3], [8, 1, 5]]
print(len(grid), len(grid[0]))
```

shows **2 3**

for row in grid:

Hands you one whole row at a time, and a loop inside it walks along that row.

```
for row in [[4, 9], [8, 1]]:
    for v in row:
        print(v)
```

shows **4 / 9 / 8 / 1**

grid[r][column]

To go down a column, keep the column number still and let the row number change.

```
grid = [[4, 9], [8, 1]]
for r in range(len(grid)):
    print(grid[r][1])
```

shows **9 / 1**

line = line + str(item) + " "

Numbers have to be made into text before they can be joined, so build the line up a number at a time and print it once at the end; the space left on the end of it is ignored when your answer is marked.

```
line = ""
for n in [2, 5, 8]:
    line = line + str(n) + " "
print(line)
```

shows **2 5 8**

Subprograms — lesson 9

def name():

Gives a block of code a name, and its indented lines do nothing until something calls that name.

```
def warn():
    print("check the seals")
warn()
```

shows **check the seals**

def name(parameter):

A parameter is a name in the brackets, and whatever you pass in arrives under that name.

```
def shout(word):
    print(word + "!")
shout("oi")
```

shows **oi!**

return value

Hands a value back to the line that called it, for the caller to print or to store.

```
def triple(n):
    return n * 3
print(triple(4))
```

shows **12**

variable = value (inside a def)

A variable first given a value inside a subprogram is local: it exists only while that subprogram is running.

```
def tidy():
    box = "new"
    print(box)
tidy()
```

shows **new**

global name

Tells a subprogram to change the variable of that name outside it instead of making a local one.

```
count = 0
def tick():
    global count
    count = count + 1
tick()
print(count)
```

shows **1**

Files — lesson 10

open(filename, "r")

"r" opens a file that is already there so that it can be read.

```
f = open("notes.txt", "r")
print(f.readline().strip())
f.close()
```

shows **red**

file.read()

Brings the whole file back as one piece of text, newlines and all.

```
f = open("notes.txt", "r") # it holds red
then blue
print("Contents:")
print(f.read())
```

shows **Contents: / red / blue /**

file.readline()

Reads only as far as the next newline, and that newline is still on the end of what comes back.

```
f = open("notes.txt", "r")
first = f.readline().strip()
f.close()
print("First line is " + first)
```

shows **First line is red**

file.readlines()

Reads the whole file into a list, with one line of it in each item.

```
f = open("notes.txt", "r")
print(f.readlines())
```

shows **['red\n', 'blue\n']**

open(filename, "w")

"w" opens a file for writing, and empties it first if it already held anything.

```
f = open("notes.txt", "w")    # notes.txt is
                               empty again
f.write("green\n")
f.close()
```

file.write(text)

Writes text into the file, adding no newline of its own, so put the \n in yourself.

```
f = open("notes.txt", "w")
f.write("green tea\n")    # the \n ends the
                           line
f.close()
```

open(filename, "a")

"a" means append: it writes onto the end and keeps everything already in the file.

```
f = open("notes.txt", "a")
f.write("green\n")    # notes.txt now ends
                       with green
f.close()
```

file.close()

Finishes with the file, and anything written is only safely in it once it is closed.

```
f = open("notes.txt", "r")
f.close()    # the link to the file has gone
```

with open(filename, "r") as f:

Does the same as open() and closes the file for you when the indented lines end.

```
with open("notes.txt", "r") as f:
    print(f.readline().strip())
```

shows **red**

text.strip()

Hands back the text with the spaces and the newline taken off each end.

```
print(len("red\n"), len("red\n".strip()))
```

shows **4 3**

text.split(",")

Breaks a line into a list of its parts, cutting it at every comma.

```
fields = "ash,blue,7".split(",")
print(fields[0], fields[2])
```

shows **ash 7**

text.split()

With nothing in the brackets it cuts at the spaces instead, which turns a typed line into a list of words.

```
words = "red blue green".split()
print(words)
```

shows **['red', 'blue', 'green']**

Swapping, slicing and printing a list – lesson 11

list[a], list[b] = list[b], list[a]

Swaps two items over in one line, which is the move every sort makes again and again.

```
pair = [9, 4]
pair[0], pair[1] = pair[1], pair[0]
print(pair)
```

shows **[4, 9]**

list[start:stop]

A slice of a list: a new list of the items from start up to but not including stop.

```
nums = [2, 5, 8, 1]
print(nums[0:2], nums[2:])
```

shows **[2, 5] [8, 1]**

" ".join(list)

Puts a list whose items are already text onto one line, with one space between each of them.

```
words = ["red", "blue", "green"]
print(" ".join(words))
```

shows **red blue green**

Checking what was typed – lesson 12

text.isdigit()

True when the text is all digits and not empty, so int() is safe to use on it.

```
print("42".isdigit(), "4.2".isdigit())
```

shows **True False**

text.isalpha()

True when the text is all letters and nothing else.

```
print("AB".isalpha(), "A1".isalpha())
```

shows **True False**

text.isalnum()

True when the text is all letters and digits, with no spaces or punctuation among them.

```
print("ab12".isalnum(), "ab 12".isalnum())
```

shows **True False**

try: ... except ValueError:

Runs the lines under try, and if they go wrong the lines under except run instead of the program stopping.

```
try:
    n = int("seven")
except ValueError:
    print("not a number")
```

shows **not a number**

if value < low or value > high:

A range check, and inclusive means the two end values are allowed, so only what is past them is refused.

```
n = 10
if n < 1 or n > 10:
    print("out of range")
else:
    print("valid")
```

shows **valid**

if len(text) < low or len(text) > high:

A length check: how many characters were typed, spaces counted in.

```
word = "spaniel"
if len(word) < 8:
    print("too short")
```

shows **too short**

while not entry.isdigit():

Asking again: read once before the loop, then keep reading while what came back still breaks the rule.

```
entry = input() # the user types ten
while not entry.isdigit():
    print("digits only")
    entry = input() # then 10
print(int(entry))
```

shows **digits only / 10**

while True:

A loop with no condition to stop it, so something inside has to decide when it has finished.

```
while True:
    colour = input() # the user types
    blue, then red
    if colour == "red":
        break
    print("not red")
```

shows **not red**

break

Leaves the loop immediately: the rest of that turn is skipped and no further turns are taken.

```
for n in [4, 9, 2]:
    if n == 9:
        break
    print(n)
```

shows **4**

Asking a table with SQL — lesson 13

SELECT column FROM table

Picks the column wanted and the table it comes from - these examples all use a table pet holding rex dog 5, mitts cat 2 and sooty cat 7.

```
SELECT name FROM pet
```

shows **rex / mitts / sooty**

SELECT column, column FROM table

Names more than one column, and they come back in the order you asked for them.

```
SELECT name, age FROM pet
```

shows **rex 5 / mitts 2 / sooty 7**

WHERE column = 'text'

Keeps only the rows that match, and text being compared against goes in single quotes.

```
SELECT name FROM pet WHERE kind = 'cat'
```

shows **mitts / sooty**

WHERE column > number

Compares a number with >, <, >= or <=, so a rule about a number need not be an exact match.

```
SELECT name FROM pet WHERE age > 4
```

shows **rex / sooty**

WHERE column <> value

<> means is not equal to, so every row except the matching ones is kept.

```
SELECT name FROM pet WHERE kind <> 'cat'
```

shows **rex**

WHERE ... AND ...

AND keeps a row only when both comparisons are true, and each side needs a comparison of its own.

```
SELECT name FROM pet WHERE kind = 'cat' AND
age > 4
```

shows **sooty**

WHERE ... OR ...

OR keeps a row when at least one of the comparisons is true.

```
SELECT name FROM pet WHERE kind = 'rat' OR
age > 4
```

shows **rex / sooty**

ORDER BY column

Sorts the rows by one column, smallest or earliest first, and comes last of all: SELECT, FROM, WHERE, then ORDER BY.

```
SELECT name, age FROM pet ORDER BY age
```

shows **mitts 2 / rex 5 / sooty 7**

ORDER BY column DESC

DESC turns the order round, so the biggest or latest comes first.

```
SELECT name, age FROM pet ORDER BY age DESC
```

shows **sooty 7 / rex 5 / mitts 2**

for row in db.execute(query):

Runs the query and hands back the rows it found, one row at a time.

```
for row in db.execute("SELECT name FROM
pet"):
    print(row[0])
```

shows **rex / mitts / sooty**

row[position]

One value out of a row, counting from 0 in the order the SELECT named the columns.

```
for row in db.execute("SELECT name, age
FROM pet"):
    print(row[0], row[1])
```

shows **rex 5 / mitts 2 / sooty 7**

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.